马云沖 · AI 产品经理作品集 · 2026.10

HR-AGENT OS
每个分数都答得上为什么

规则算出 ·
可复现 ·
答得上「为什么」

(50+) 岗位模板
人才画像库
全链路闭环，

运行时依赖
0 个 npm 包

分档口径
100% 规则算出 · 可复现

误伤率 · 红线
0% —— 宁可多聊一轮

规模
24 页面 · 47 接口 · 19,400 行 JS

点击画面 · 翻到下一页

PRODUCT
9 人在投

A DAY IN HR

场景还原 —— 取材自法务实务与工程实测，非用户访谈记录

09:12
邮箱里躺着 80+ 份未读简历

10:40
第 3 次被打断 —— 有人来问社保

13:20
翻到第 41 份，已经记不清前面看过谁

15:05
JD 里一句不合规的话，写的人没觉得有问题

16:30
被问「这个分怎么来的」—— 答不上来

18:40
周报还没拼完。而明天，还是 80+ 份。

马云冲 · AI 产品经理作品集 · 2026.10

一天里，80% 的活，不需要判断。

翻 80 份简历 3–4 小时 · 被问社保打断 20 次 · 每月手工拼周报 4 小时。这不是能力问题，是工具问题 —— 而这些活，恰好是 AI 最该接住的。

HR-Agent OS

招聘全链路 AI Agent 平台 · 让每个分数都答得上「为什么」

FRONTEND
6 人在投

DESIGN
7 人在投

DATA
8 人在投

运行时依赖	分档口径	误伤率 · 红线	规模
0 个 npm 包	100% 规则算出 · 可复现	0% —— 宁可多聊一轮	24 页面 · 47 接口 · 19,400 行 JS

点击画面 · 翻到下一页

02 CONTENTS

目录

五个章节，正好对应使用者最常追问的五个问题

01 我在解决什么问题 三条真实痛点，每一处设计都对应其中一条 P.05

02 产品设计 三个「必须」与三个「不做」 P.07

03 怎么证明它有效 一套自己搭的离线评测 P.10

04 工程与过程 让文档不许撤谎 P.13

05 自审与边界 我知道它哪儿还不行 —— 12 项自审清单与已知边界 P.17

03

01 / OVERVIEW

30 秒速览

如果只有半分钟，看这一页

这是一款能被追问的 B 端 AI 产品：分数 100% 由确定性规则算出（可复现、可追溯），权限在服务端真拦截（真 403），JD 会对着真实在招市场接地并做合规扫描。**原型是「点得动」，这个项目是「答得上」。**

24

页面 · 47 个后端接口

schema v9 · 12 个回归套件

约 19,400 行 JS

96.7%

档位一致率 · 29 / 30

±1 档 100%

n = 30 · rubric v1 · 人工标注

0%

误筛率 · 红线

宁可多聊一轮

不可错杀一份

0

个 NPM 包

Node 22 原生 http

+ 内置 node:sqlite

我担任

产品定义 + 口径设计 + 工程落地 + 自建评测

（独立完成）

给谁做 · 真实数据底座

中大型企业 HR 部门 / 招聘团队

私有化部署 · 数据不出境

183 条市场在招岗位（BOSS 147 · 智联 36）

21 个岗位 · 79 份候选人 · 9 个演示账号

在线 DEMO

https://myc20050802-debug.github.io/hr-agent-os

零安装，点开就玩

PRODUCT
7 人在投

FRONTEND
7 人在投

SECTION 01

我在解决什么问题

HR 场景里，AI 落地真正难的不是「接一个大模型」，
而是让它给出的结论，能被业务方拿去签字。

DESIGN
10 人在投

DATA
6 人在投

05

02 / PAIN POINTS

三条真实痛点

每一处设计都对应其中一条 —— 依据是法务实务与工程实测，不是凭空假设

01

结论不敢签字

分数是黑盒，出了问题无法追溯 —— 业务方凭什么签？

依据 · 招聘流程实务：争议发生在「凭什么信」，不是「准不准」

→ 解法

分数 100% 由确定性规则算出（可复现、可追溯）；分数下面常驻一栏 「为什么是这个分」 —— 逐维算式 + 决定性因素 + 还差几分进下一档。

02

敏感数据没权限

权限只写在页面上，数据早已出了库 —— 等于没权限。

依据 · 可当场复现：员工身份导出全公司花名册 → 真 403

→ 解法

权限 在出数据之前生效：24 能力 × 7 角色 + 行级范围（all / dept / self）+ PII 物理脱敏； 越权是 真 403，前端按钮显隐只是体验层。

03

JD 即法律风险

一句涉及年龄、性别或户籍的老措辞，写的人不觉得有问题，HR 也未必逐句审。

依据 · 执业法律背景 +《就业促进法》第 27 条（举证责任在企业）

→ 解法

生成前检索市场在招同类岗位「接地」，生成后做合规扫描 —— 命中就业歧视性表述（年龄／性别／户籍等） 直接阻止发布，每条给法条依据 + 建议改法。

目标用户

中大型企业 HR 部门 / 招聘团队（私有化、数据不出境）

PRODUCT
7 人在线

FRONTEND
7 人在线

SECTION 02

产品设计

先定不可妥协的边界，再谈功能。
我把价值观写成了可被检验的工程约束。

DESIGN
12 人在线

DATA
4 人在线

02

07

03 / DESIGN RULES

三个「必须」× 三个「不做」

先定不可妥协的边界，再谈功能

PRODUCT 1 人在线

FRONTEND 2 人在线

MUST · 必须做到

必须可解释

每个分数都能摊开算给业务方看；归因只读打分时已算出的系数、不做二次计算，测试锁死「分项之和 == 总分」。

必须可复现

同一份简历跑两次，结果必须一致 —— 否则不能拿去和业务方对齐。

DESIGN 7 人在线

必须留痕

应用层只插不改，外加数据库触发器 RAISE(ABORT) —— 想改审计，得先改 schema，而 schema 是版本化迁移管的。

WON'T · 明确不做

不把判断权交给模型

分数与档位全由确定性规则算出；模型只做措辞、语言组织、JD 润色。

不编硬数字

成本与用量必须能指出来源；规则模式就是 0 调用、显示 0。同理 —— 没有用户数据就说没有，**不用构造样本充数**。

DATA 9 人在线

不让 AI 越过人工闸门

Offer 决策、推翻 AI 结论这类高风险动作，必须留人工签批。

DESIGN RULES

08

04 / CAPABILITIES

五个核心能力

每条都写明「为什么它是真的」—— 不是话术

01	**分数可解释**	逐维算式（技能匹配 40 × 0.95 = 38）+ 决定性因素 + 还差几分进下一档；测试锁死「分项之和 == 总分」「满分 × 系数 == 该维得分」。
02	**规则与模型分工明确**	分档 100% 规则（可复现、可签字）；模型只做措辞与语言组织 —— 「不配大模型也能跑通全流程」是设计选择，不是降级方案。
03	**权限真的在服务端**	24 能力 × 7 角色 + 行级范围 + PII 脱敏，全部在出数据之前生效；员工身份导出全公司花名册是 真 403，前端按钮显隐只是体验层。
04	**审计改不掉**	应用层只插不改 + 数据库触发器 RAISE(ABORT) —— 审计日志无法被应用层篡改。
05	**JD 接地真实市场 + 合规红线**	按「标题优先」检索市场在招同类岗位（真实抓取 183 条：BOSS 147 · 智联 36，69 条含完整 JD 正文、36 条带真实薪资）；命中就业歧视性表述（年龄／性别／户籍等） 直接阻止发布，每条给法条依据与建议改法。

CAPABILITIES

PRODUCT
11 人在投

FRONTEND
9 人在投

SECTION 03

怎么证明它有效

AI 产品第一个要说服人的问题，通常是「你怎么衡量效果」。
所以我自己搭了一套评测 —— 它否决过我的方案，也照出过我改不动的地方。

DESIGN
6 人在投

DATA
4 人在投

03

10

05 / EVALUATION

黄金集与标注口径

标准答案该怎么造，才能让人信 —— 使用者一定会追问「你这个集子怎么来的」

01	**规模**	30 例（3 个岗位 × 各 10 例），人工标注，三档标签 strong / ok / no
02	**判定顺序**	先过硬性门槛（年限 / 学历 / 技能相关性），再定档位 —— 顺序反了，会把「技能很全但年限不够」标成优秀。
03	**证据来源**	只看简历原文 + 岗位要求，**禁止偷看算法输出** —— 看一眼，就等于把规则抄成了答案。
04	**口径冻结**	rubricVersion = 1 写进代码、可机器校验：枚举合法 / id 唯一 / jobId 无悬空 / 每岗位 ⩾10 例 / 每岗 ⩾1 陷阱探针 / 三档占比 ∈[20%,50%]。
05	**扩样方案**	独立成文：7 个必须先拍死的决策、三层难度配额、一致性验证 —— 不拿规则输出当答案。

标准答案必须同时具备三个属性 —— **独立性** · **稳定性** · **够难**；缺一个，评测数字就变成「自证」

GOLDEN SET

11

06 / METRICS · N = 30 · RUBRIC V1

三个指标与一条红线

说得出指标背后的业务取舍，比报一个漂亮数字更有说服力

96.7%

METRIC 01 · 档位一致率

29 / 30，严格三档一致

100%

METRIC 02 · ±1 档一致率

从没出现「差两档」的离谱判断

0.0% 红线

METRIC 03 · 误筛率

把好简历判死的代价
远高于把差简历放进面试

8.3%

COUNTERPART · 漏筛率

12 个应拒样本里放了 1 个进来
—— 宁可多聊一轮，不可错杀

统计诚实

样本只有 30 例 —— 所以不只报点估计：同时给出 Wilson 95% 置信区间 [83.3%, 99.4%]。主动暴露区间偏宽，比藏起来更可信。

门槛拦截 9 例

年限 ×2 · 学历 ×1 · 技能零命中 ×6 —— 这些在看分数之前就被拦掉了。

度量盲区 · 主动说

JD 生成质量没有客观指标 —— 只有筛选侧有黄金集。这一块我不拿「看起来不错」当结论，它列在已知边界里。

METRICS

12

07 / ITERATION

一次真实的迭代

评测驱动，不是拍脑袋调阈值 —— 我改的是「判定链条」，不是「业务维度计数」

档位一致率	~~90.0%~~ →	**96.7%**	29 / 30，严格三档一致
漏筛率	~~25.0%~~ →	**8.3%**	应拒样本放进来的比例，大幅下降
误筛率 红线	0% →	**0%**	守住 —— 这是选方案时的硬约束，不是可选项

怎么选的

5 个候选方案逐一实测，只有这一个同时满足「一致率不降 **且** 误筛不涨」。它带来的是 一致率 +6.7pt · 漏筛 −16.7pt · 误筛不动 —— 三个数出自同一次改动、同一次跑分，不是分别调出来的。

被否决的方案

「业务维度与职能族词表求交」看起来更聪明，实测一致率仅 73.3%、误筛 0% → 22.2%，直接破了红线 —— 因此明确否决。

评测的作用不是给一个分数，而是 **帮我否决看起来更「聪明」的方案**

13

08 / ARCHITECTURE

技术实现概览

每一行都带一条「为什么这么选」—— 技术选型也是产品决策

L5	**运行时**	Node 22 原生 http + 内置 node:sqlite → 零依赖，clone 下来一条命令就能起，PoC 阶段把「环境问题」这个变量彻底消掉。
L4	**前端**	单文件 HTML（内联全部 CSS/JS）→ 断网可演示、双击即开；代价我主动说：没法按需加载、没有构建期校验。
L3	**分层**	后端 16 个模块按 L0–L5 分层 → 依赖方向单向，改一层不会牵连全栈。
L2	**来源单一**	shared/ 三个文件是前后端共用数据源 → 改一处，离线端与后端同时生效，避免两套逻辑漂移。
L1	**数据库**	schema v9，版本化迁移 → 空库自动迁移 + 种子，不提交任何数据文件。

24 页面 · 47 接口 · 约 19,400 行 JS · 10,300+ 行文档

全部由脚本从代码算出 —— 写进文档的数字，有守卫盯着

你自己跑一遍 npm start · npm test（12 套件） · node tools/eval_dual_mode.js（评测，零配置）

ARCHITECTURE

14

09 / ENGINEERING

让文档不许撒谎

文档里的每个数字，都由程序自己核对过 —— 不是「我写完了」，是「我赖不掉」

1	改一处 · 全部重跑	12 组测试一条命令跑完 —— 不会「修好这里、弄坏那里」
2	数字 · 机器替你对	从代码里数出 24 页面 / 47 接口 / 545,169 字节，与文档逐个比 —— 对不上就报错
3	旧数据 · 不许复活	不只查现在写对没，还查改过的地方有没有变回老数据
4	分数 · 不许倒退	效果指标存一条基准线，一旦退步就卡住，过不了关
5	网页 · 一条命令重生	改了文字，阅读版网页一条命令重生 —— 不会新旧两版并存
6	换机器 · 照样能跑	两个 Node 版本各跑一遍 + Python 语法检查 —— 不靠「我机器上能跑」

案例 01 · 一次误判

6 项守则 · 全绿

01 我曾把评测基线，写成
「四舍五入后」的值 —
而守则是单向往上比的。

于是
真值 96.67% vs 基线 96.7%
被程序判成「退步」，
报了一次误判。

修法：基线只存原始值，
不存四舍五入后的结果。
教训已写进贡献指南。

DISCIPLINE

15

10 / PROCESS

从 0 到 1 的七个阶段

每个阶段的产出物都留在仓库里 —— 过程是可查的，不是口述的

01	**立项**	定位 · 目标客户 · KPI 树 · Non-goals → 产品需求草稿
02	**需求调研**	岗位词库（职能族 / 行业 / 技能尾巴）+ 市场岗位快照 → 183 条市场岗位
03	**产品设计**	口径设计：打分维度与权重、判定链条、JD 生成与合规规则 → v13 / v14 / v16 修复说明
04	**开发**	分层骨架、权限模型、状态机、审计触发器 → 24 页面 / 47 接口
05	**测试**	契约测试 → 质量评测 → 压测 → 文档守卫 → 12 套件 + 黄金集 + 基线守卫
06	**上线**	上线自检、备份恢复、三档可用性路径 → 上线手册、部署路径文档
07	**运营准备**	自审：先当一遍最挑剔的用户，翻自己的代码 → 缺口审计清单

16

11 / INCIDENTS

三次真实翻车

每一次都留了回归用例 —— 演进是真的，不是一次写成

INCIDENT 01

「测试全绿」其实是假的

根因

前端套件把页面 id 写死成数组，新增页面时静默漏测，却照打「全部通过」。

修法

改为从页面注册表派生，并由守卫禁止再写死。

→ tools/test_nav.js + 文档守卫不变量

INCIDENT 02

离线端与后端逻辑漂移

根因

同一个判定在两处实现，改了一处、忘了另一处。

修法

抽 shared/ 单一数据源；离线端必须镜像同一份词库与口径，改完跑对应套件。

→ tools/test_jd.js（改一处必须同改两处，否则红）

INCIDENT 03

规则模式下却报了 token 与成本

根因

想让演示「看起来完整」，于是显示了一个凭空算出的金额。

修法

规则模式一律显示 0，成本卡改「试算」并注明来源。

→ tools/test_screening.js（规则模式 token 必须为 0）

一个 AI 产品如果连自己的成本数字都不可信，业务方 **不会信它的任何输出**

INCIDENTS

17

12 / SELF-AUDIT

我审了一遍自己的项目

交付之前，我先当了一遍最挑剔的用户 —— 按「用户会先撞到哪一处」排序，而不是按修复难度

PRODUCT

FRONTEND
4 人在投

12

项缺口审计

全部写在 docs/ 里，可逐条核对；已修的补了回归用例，你可以自己验。

其中一条，我决定不修

「业务匹配」只数标签个数 —— 会让「资深销售」这类无关背景被抬到 ok（实测 77 分）。我按评测口径跑了一遍修法：一致率掉到 73.3%、误筛 0% → 22.2%，破了红线 —— 于是否决修法、保留原实现，并把「为什么不修」写进口径文档。

DESIGN
9 人在投

虚报模型调用与成本

筛选链路里其实 0 次模型调用，日志却记了模型工具名和 token 数。

✓已修：工具名不撒谎，用量取自真实网关

「已进入优化数据集」是空话

审计日志这么写，但那个数据集并不存在 —— 纯话术。

✓已修：改成可归因的原因码枚举 + 一致率接口

一个死按钮 + WAL 只增不减

演示复跑必经路径点下去毫无反应；WAL 日志实测是主库的 11 倍，且从不 checkpoint。

✓已修：补后端接口；压测时顺手记录 P95 / P99

无障碍为 0

政府采购评标普遍含无障碍要求，而它一项都没做。

✓已补最小集：弹窗语义 / 焦点管理 / 表格表头 / 减少动效

DATA
12 人在投

产品文档习惯只写「已完成」；我把「还没做好、以及我为什么知道」也摊开 —— **缺陷由用户上线后自己撞见，比我先说贵得多**

SELF-AUDIT

18

13 / KNOWN LIMITS

已知边界（不藏着）

说得清边界，比假装没有边界更有说服力

known-limitations.md

6 ITEMS · OPENLY LISTED

01	**黄金集 30 例**	置信区间偏宽 [83.3%, 99.4%] · 扩样必须独立标注才有效
02	**单租户**	租户标识写死 · 多租户涉及数据隔离模型重设计，属企业级门槛，不在 PoC 范围
03	**无真实简历入口**	目前走演示数据 · 真实简历涉及个人信息合规，需先定数据来源与授权链路
04	**模型未接入默认链路**	规则模式跑全流程 · 这是设计选择（可复现、可签字）；接模型是替换一个函数，不改架构
05	**并发天花板未压到极限**	已测到 20 并发 0 错误 · SQLite 单写者，需在真实部署形态下再压
06	**无障碍仅最小集**	已补语义与焦点管理 · 完整 WCAG 审计未做，不能当合规

19

14 / ABOUT ME

关于我

马云冲 · AI 产品经理

“

一份 JD 或 Offer 里哪句话碰不得，
我是从法条那一侧知道的。

我对使用者的三条承诺

01 「分数和档位 100% 是规则算出来的，模型只负责措辞。」

02 「误筛率 0% 是我守的红线；为了它，我接受了 8.3% 的漏筛。」

漏筛：不合适的人混进了面试 · 误筛：合适的人被错拒

03 「我知道它哪儿还不行 —— 这份自审清单就是证据。」

工作方式

AI 写代码，但我能解释每一行为什么这么写 —— 招聘结论你要拿去用，它就得经得起追问。

在线 DEMO（打开就能用）

https://myc20050802-debug.github.io/hr-agent-os

源码 + 完整文档

https://github.com/myc20050802-debug/hr-agent-os

自己验一遍（零配置）

npm start · npm test · eval_dual_mode.js